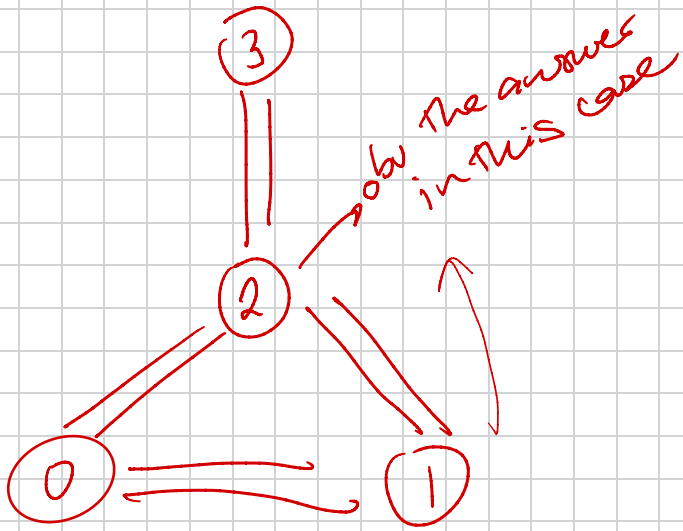
Noisy QAOA (Quantum Approximate optimization Algorithm)

- applied to minimum vertex cover

For this prob

$$H_{cost} = \frac{3}{4} \sum_{(i,j) \in edges} (Z_i Z_j + Z_i + Z_j) - \sum_{i \in vertices} Z_i$$



$$H_{mixer} = \sum_{i \in vertices} X_i$$

- A cost hamiltonian: quantum operator used in quantum computing/ quantum optimization algos that translates a classical optim. problem into the language of quantum mechanics.
  - encodes objective: maps the variables of a classical problem onto qubits using Pauli operators (typically Pauli Z).
  - ground state is the solution: usually engineered such that the lowest possible energy state corresponds to the best solution.

$$H_{cost} = \frac{3}{4} \sum_{(i,j) \in edges} (Z_i Z_j + Z_i + Z_j) - \sum_{i \in vertices} Z_i$$

  - here: the quantum state of each qubit represents a vertex. $Z_i = -1$ means vertex i is included in cover.
    $Z_i = +1$ means excluded
  - broken down into two components:
    the edge penalty & the vertex minimization weight

$$H_{cost} = \frac{3}{4}\sum_{(i,j)\in edges}(Z_iZ_j + Z_i + Z_j) - \sum_{i\in vertices} Z_i$$

- The Edge Penalty term: $\frac{3}{4}\sum_{i,j\in edges}(Z_iZ_j + Z_i + Z_j)$

Purpose: enforces constraint: every edge must be covered by a vertex
- For an edge (i,j):
- if both vertices in cover $(Z_i = -1, Z_j = -1)$: $(-1)(-1) - 1 - 1 : 1 - 2 : -1$ (valid & lowest energy)
- one vertex in cover $(Z_i = -1, Z_j = 1)$: $(-1)(1) - 1 + 1 : -1 - 1 + 1 : -1$ (valid & lowest energy)
- No vertex in cover $(Z_i = 1, Z_j = -1)$: $(1)(1) + 1 + 1 : +3$ (invalid)
  - heavily penalizes system's total energy

- The Vertex Term (objective Function): $-\sum_{i\in vertices} Z_i$

Purpose: minimizes total # of vertices chosen for the cover.

term = $-1($num_vertices_included(-1) + num_vertices_excluded$(1))$

= included − excluded ∴ ↑ excluded favors a minimized energy level.

- 3/4 penalty scaling factor so that the edge constraint is more imp. than trying to reduce vertex count.

A mixer Hamiltonian: quantum operator drives exploration. injects fluctuations to explore the solution space.
- prevents getting stuck: with only H_{cost}, it immediately get stuck in local minimas.
- creates superposition: induces quantum tunneling & transitions btwn classical states, enabling the algorithm to search many possibilities simultaneously.

- The Standard X-mixer:
  - most common for problems with binary variables (qubits) is the transverse-field mixer,

$$H_m = \sum_{i\in vertices} X_i$$

- Pauli X op acts as a bit flip ($|0\rangle \leftrightarrow |1\rangle$)

QAOA

In QAOA, the algo alternates btwn apply the H_{cost} and H_{mixer} in blocks.

$$U(H_C, \gamma) = e^{-i\gamma H_C} \quad \text{and} \quad U(H_M, \beta) = e^{-i\beta H_M}$$

- Cost layer: adds phase to states based on how good or bad sol. is.
- Mixer layer: blends states together, allowing interference to happen so that good states amplify & bad cancel out.

- After cost H, objective value is encoded as quantum phase (rotation around Z axis) on each state.

- If u measured now, the phase disappears & u get random results. Z-phase doesn't change probs. on its own.

- Say u begin with state/vector in $|+\rangle$ state (along x-axis/equator).
  - The H_{cost}: spins it along the equator
  - The H_{mixer}: pulls it off the equator
    - if nearer to Y-axis, pulls up toward North pole ($|0\rangle$)
    - if nearer to X-axis, " " South Pole ($|1\rangle$)

The Math:

- initial state: $|\psi_0\rangle = |+\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix}$
- Apply cost H (Z-rotation):

$$U_Z(\phi) = e^{-i\frac{\phi}{2}Z} = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix}$$

$$|\psi_1\rangle = U_Z(\phi)\,|+\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} e^{-i\phi/2} \\ e^{i\phi/2} \end{pmatrix}$$ (or ψ_0) — measuring now is still 50/50

- Apply Mixer H (X-rotation):

$$U_X(\theta) = e^{-i\frac{\theta}{2}X} = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos\frac{\theta}{2} \end{pmatrix}$$

P(0) & P(1) if in state $|0\rangle$

$$|\psi_2\rangle = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos\frac{\theta}{2} \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} e^{-i\phi/2} \\ e^{i\phi/2} \end{pmatrix}$$

↑ P(0) and P(1) if in state $|0\rangle$

- Focusing on just the top component

$$\text{Amplitude}(|0\rangle) = \frac{1}{\sqrt{2}}\left[\cos\tfrac{\theta}{2}\cdot e^{-i\phi/2} - i\sin(\tfrac{\theta}{2})\,e^{+i\phi/2}\right]$$

NB: $e^{ix} = \cos x + i\sin x$

$$= \frac{1}{\sqrt{2}}\left[\cos\tfrac{\theta}{2}\left(\cos\tfrac{\phi}{2} - i\sin\tfrac{\phi}{2}\right) - i\sin\tfrac{\theta}{2}\left(\cos\tfrac{\phi}{2} + i\sin\tfrac{\phi}{2}\right)\right]$$

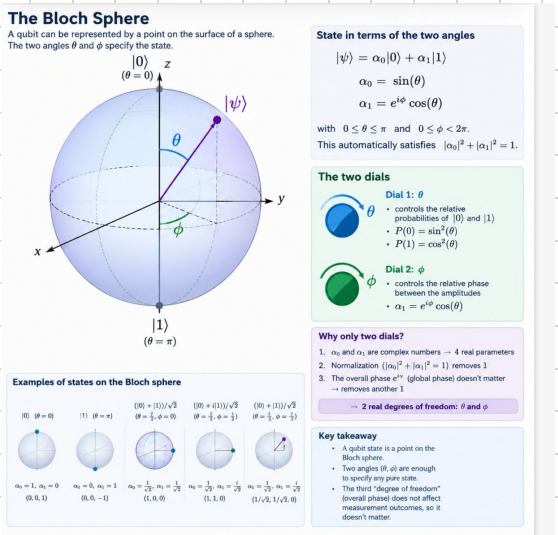
$$= \frac{1}{\sqrt{2}}\left[\cos\tfrac{\theta}{2}\cos\tfrac{\phi}{2} + \sin\tfrac{\theta}{2}\sin\tfrac{\phi}{2} - i\left(\cos\tfrac{\theta}{2}\sin\tfrac{\phi}{2} + \cos\tfrac{\theta}{2}\sin\tfrac{\theta}{2}\right)\right]$$

trig identities

$$= \frac{1}{\sqrt{2}}\left[\cos\left(\frac{\theta-\phi}{2}\right) - i\sin\left(\frac{\theta+\phi}{2}\right)\right]$$

Then $P(0) = |\text{Amplitude}(|0\rangle)|^2 = \frac{1}{2}\left[\cos^2\left(\frac{\theta+\phi}{2}\right) + \sin^2\left(\frac{\theta+\phi}{2}\right)\right]$

$$= \frac{1}{2}(1 + \sin\theta\cos\phi)$$



So $P(0) = \frac{1}{2}(1 + \sin\theta\cos\phi)$

- θ is the mixer knob
- ϕ cost knob

if $\phi = 90°$ (+Y axis): $\sin(90°) = 1$

$P(0) = \frac{1}{2}(1 + \sin\theta) = 1$ if $\theta = 90°$

$\theta > 0$, ↑ $P(0)$

if $\phi = -90°$ (−Y axis): $\sin(-90°) = -1$

$P(0) = \frac{1}{2}(1 - \sin\theta) = 0$ if $\theta = 90°$

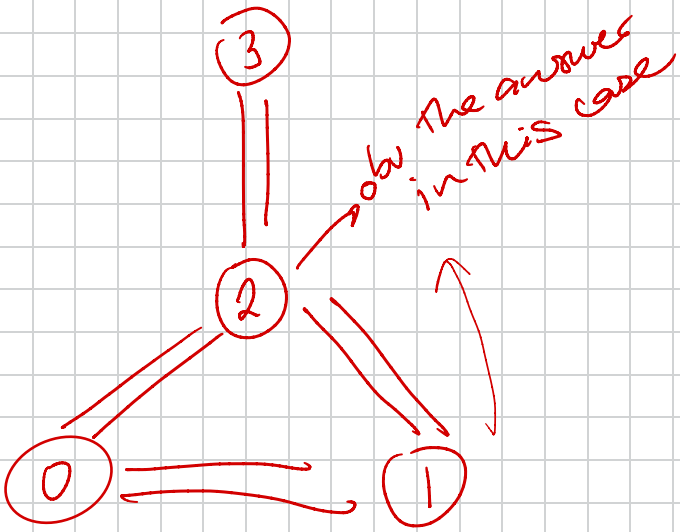
$\theta > 0$, ↓ $P(0)$

if $\phi = 0$ (X-axis): $\sin(0) = 0$

$P(0) = \frac{1}{2}(1 + 0) =$ 50/50 no matter the mixer knob.

Summary Table for your Mental Model

Algebraic Syntax	Physical Meaning	Quantum Circuit Implementation
$A + B$	Separate energy contributions	Implemented sequentially (one gate after another)
$Constant \times A$	Penalty scaling / Priority weight	Modifies the **rotation angle** (θ) of the gate
Z_iZ_j	Variable interaction (Correlation)	Requires **CNOT** gates to entangle Qubit i and Qubit j



$$H_{cost} = \frac{3}{4} \sum_{(i,j) \in edges} (Z_iZ_j + Z_i + Z_j) - \sum_{i \in vertices} Z_i$$

$$H_{mixer} = \sum_{i \in vertices} X_i$$

Cost of true min

vertices 2 and 1

$$H_{cost} = \frac{3}{4}(-1 - 1 - 1 - 1) - (1 - 1 - 1 + 1)$$

$$= \frac{3}{4}(-4) = -3$$